Kubernetes　Chapter 11

Probe・HPA・ログで本番運用を安全にする

可観測性の3本柱でKubernetesの運用品質を高める

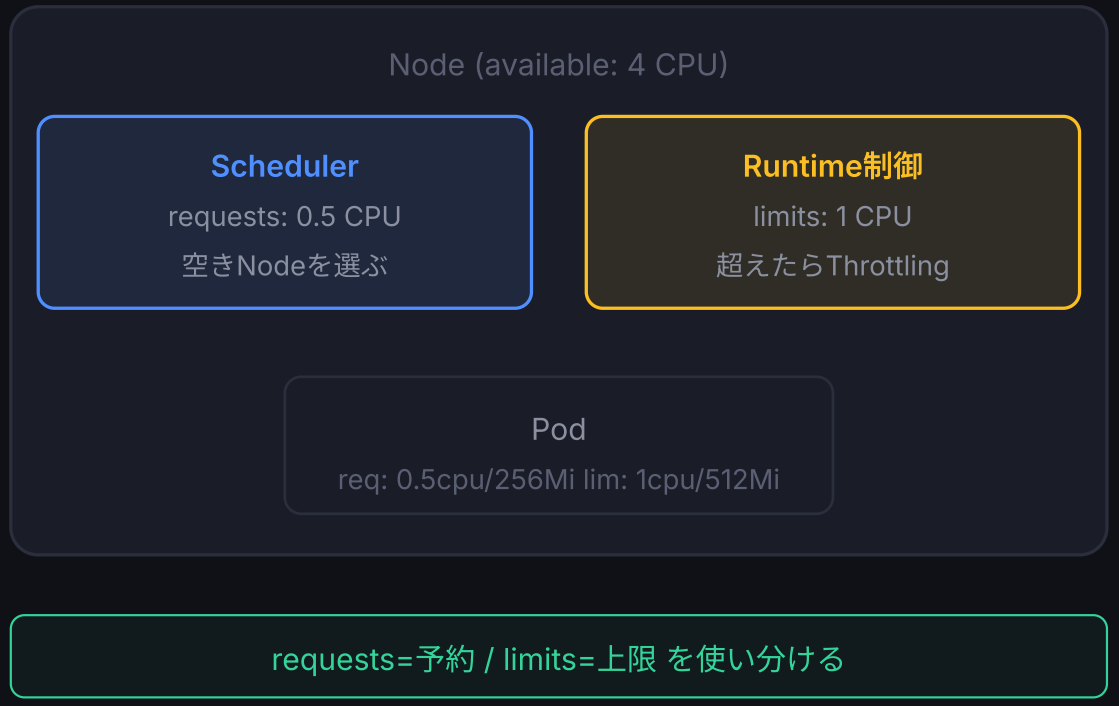
SECTION 01

リソース管理と
オートスケール

requests/limitsでスケジューリングと実行制御を分離する

- **requests**はスケジューラが使う。「このPodを動かすには最低このリソースが必要」という宣言
- **limits**は実行時の上限。CPU throttlingとOOMKillでコンテナを制御する
- requests未設定だと**Nodeが過負荷になってもスケジュールされ続ける**。必ず設定する

ECSのtask definitionのcpuとmemoryの設定に相当する

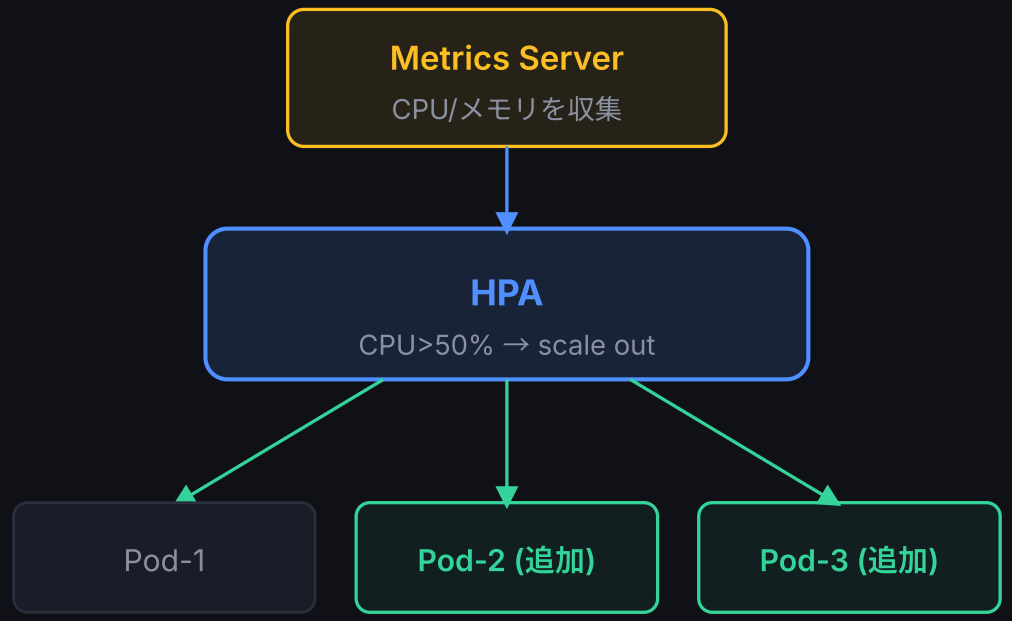


requests=予約 / limits=上限 を使い分ける

requests vs limits — 予約 vs 上限

観点	REQUESTS（予約）	LIMITS（上限）
使われるタイミング	スケジュール時（Pod配置判断）	実行時（実際のリソース制御）
CPU超過時	（影響なし）	Throttling（速度制限）
メモリ超過時	（影響なし）	OOMKill（コンテナ強制終了）
未設定のリスク	Node過負荷でも配置される	リソースを際限なく使用できる

HPAがMetrics Serverのメトリクスで自動スケール



- HPA（Horizontal Pod Autoscaler）はECSのオートスケーリングポリシーに相当。CPU使用率等でreplicas数を自動調整する
- HPAはMetrics Serverからメトリクスを取得する。**Metrics ServerはMinikubeではaddonで有効化**する
- targetCPUUtilizationPercentage: 50 なら**平均CPU使用率が50%を超えたらPodを追加**し、下がったら削減する

SECTION 02

可観測性とProbe

Readiness / Liveness / Startup — 3種のProbeの使い分け

PROBE種別	目的	失敗時の動作
Readiness	Podがトラフィックを受け入れられるか確認	Endpointsから除外（トラフィックが届かなくなる）
Liveness	Podが正常に動作しているか定期確認	コンテナを再起動（デッドロック等の自動回復）
Startup	起動に時間がかかるアプリの初期化完了待ち	LivenessProbeより先に実行。完了するまでLivenessをブロック

sternで複数Podのログをリアルタイム Tail

- kubectl logsは**1Pod1コンテナずつしか見られない**。Deploymentで複数Pod動いていると手間がかかる
- **stern**はPod名のパターンマッチで複数Podのログを同時に色分け表示できる
- コンテナログは**/var/log/containersにファイルとして保存**される。FluentdなどがこれをCloudWatch/Datadogに転送する

stern api-pod --since 5m → api-podで始まる全Podの直近5分のログ

```
$ stern api-pod --since 5m
```

```
api-pod-1      GET /health 200
api-pod-2      POST /users 201
api-pod-1      ERROR db timeout
api-pod-3      GET /items 200
api-pod-2      DELETE /users/5 204
...
```

複数Podのログが色分けで一覧表示される

まとめ

01

requests/limitsは必ず設定

requestsでスケジューリング、limitsで実行制御を分離する。未設定はNode過負荷やリソース枯渇の原因になる

02

3種のProbeで安全性を担保

ReadinessでトラフィックをブロックしてLivenessでゾンビPodを回復する。起動が遅いアプリにはStartupを使う

03

HPAとsternで運用を自動化

HPAがCPU使用率に応じてPod数を自動調整。sternで複数Podのログをリアルタイム Tailして障害を素早く検知する